Deep Debug: investigar com prova, não com palpite

Você não sabe. Você verifica. Esta skill existe porque um palpite plausível sobre produção custa mais caro que dez minutos de consulta.

Regra zero (não negociável)

Nenhuma afirmação sobre o comportamento do sistema sai sem evidência anexada.

Toda frase que você escrever cai numa destas categorias, e você **rotula** cada uma:

Rótulo	Significa	Pode virar achado?
`[COMPROVADO]`	Tem consulta, log, diff ou arquivo:linha anexado	Sim
`[HIPÓTESE]`	Plausível, ainda não checado. Vem com o comando que checaria	Não. Vira tarefa
`[NÃO VERIFICÁVEL]`	Fonte indisponível (sem acesso, sem retenção, sem credencial)	Não. Declare o bloqueio

Proibido: "provavelmente", "deve estar", "normalmente esse tipo de erro", "imagino que", "pela minha experiência", sem um desses rótulos ao lado. Se não checou, diga que não checou.

Se não deu para comprovar, a resposta certa é "não consegui comprovar X, preciso de Y", e não um achado com cara de certeza.

Fontes obrigatórias

Varra todas as que forem relevantes antes de concluir. Não pare na primeira: uma hipótese só fecha quando **código, dado e log** concordam.

Antes de usar a skill pela primeira vez, preencha a seção "Mapa do meu sistema" lá embaixo. Ela é o que transforma este método genérico em algo que a IA consegue executar no seu ambiente.

1. O código que está rodando (não o que está no seu disco)

O que está na sua máquina não é necessariamente o que rodou em produção.

```
git log -1 --oneline                        # o que você está lendo
gh pr view <n> --json headRefOid,files      # o que o PR realmente mexe
# o que está deployado (exemplos; use o do seu ambiente):
kubectl get deploy <app> -n <namespace> -o
jsonpath='{.spec.template.spec.containers[0].image}'
curl -s https://<seu-servico>/version
```

2. Banco de dados: o dado de produção

```
# exemplos; use o cliente do seu banco
psql "$PROD_READONLY_URL" -c "SELECT count(*) FROM <tabela> WHERE <filtro>;"
bq query --use_legacy_sql=false --max_rows=1000 "SELECT ..."
```

Sempre rode um `COUNT(*)` antes da consulta de detalhe. Muitos clientes limitam as linhas retornadas e **cortam em silêncio** (o `bq`, por exemplo, devolve 100 linhas por padrão). Conclusão tirada de resultado cortado é palpite disfarçado de dado.

Produção é somente leitura nesta skill. Nada de `INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER` ou `CREATE OR REPLACE` em produção. Antes de rodar qualquer consulta, **mostre a consulta e espere o "pode"**. Escrita só em homologação ou sandbox, e com pedido explícito.

3. Logs: o que realmente aconteceu

```
# exemplos
kubectl logs <pod> -n <namespace> --since=2h
gcloud logging read '<filtro>' --project=<projeto> --freshness=7d --limit=500
aws logs filter-log-events --log-group-name <grupo> --start-time <epoch_ms>
```

Passe sempre o projeto, a conta ou o ambiente de forma explícita: o padrão da sua linha de comando quase nunca é produção.

⚠️ Nível "ERROR" no agregador de logs costuma marcar **qualquer coisa escrita em stderr**: avisos de biblioteca, instalador de pacote, logger mal configurado. Erro de verdade é stack trace mais a mensagem de falha do job. Não reporte o nível do log como se fosse a falha.

⚠️ Processo que morre não reporta. Se o job foi derrubado (falta de memória, tempo esgotado, processo recolhido), o log dele pode simplesmente parar no meio. Procure o motivo no orquestrador, não só no log da aplicação.

4. Ambiente e configuração

```
kubectl exec -n <namespace> deploy/<app> -- env | grep -E "<PREFIXO_DAS_SUAS_VARIAVEIS>"
kubectl get pods -n <namespace>
```

Leia a configuração que **de fato** está no ambiente antes de afirmar o que rodou. Arquivo de configuração no repositório não é prova de configuração em produção.

5. CI e PR

```
gh pr checks <n>
gh run view <id> --log-failed
```

Procedimento

1. **Escreva a pergunta numa frase.** "Por que o job X não processou o registro Y na execução Z", e não "investigar o job".
2. **Liste as hipóteses** (de 2 a 4) e, para cada uma, **qual fonte mata ou confirma**. Mostre essa lista antes de sair consultando.

3. **Colete evidência** da fonte mais barata para a mais cara. Guarde os resultados numa pasta de rascunho, fora do repositório.
4. **Triangule.** Código diz A, dado diz B, log diz C: se divergirem, a divergência *é* o achado. Não escolha a narrativa mais bonita.
5. **Reporte** no formato abaixo. Hipótese não confirmada morre ou vira tarefa explícita. Nunca vira conclusão.

Formato de saída

```
## Pergunta
<uma frase>

## Veredito
[COMPROVADO] <o que é fato, com o número ou a linha que prova>

## Evidências
| # | Afirmação | Fonte | Comando/consulta | Resultado |
|---|---|---|---|---|

## Não verificado
- <hipótese> — mataria com: <comando>. Bloqueio: <acesso / retenção / credencial>
```

Em code review

Só entra no comentário de review o que estiver `[COMPROVADO]`. Suspeita não comprovada, se valer a pena, vira pergunta ao autor ("rodei X e vi Y, isso é esperado?"), nunca afirmação de bug.

Armadilhas comuns (comece por estas e acrescente as suas)

- **Coluna que some sem erro.** Rotina de escrita que filtra o dataframe pelas colunas do schema descarta o resto calada. Campo faltando em produção: confira o schema da tabela antes de culpar quem gera o dado.
- **Deploy que não chegou.** Sincronização de código ou de imagem que falha em silêncio deixa o ambiente numa versão antiga. Confira a versão que está rodando (fonte 1) antes de debugar a versão nova.
- **Resultado cortado.** Ver a fonte 2: sem `COUNT(*)`, você não sabe se viu tudo.
- **Filtro que esvazia em silêncio.** Junção com uma tabela de referência que não existe ou veio vazia zera o resultado sem erro nenhum.
- **"Não aconteceu" que na verdade "já tinha acontecido".** Etapa anterior que já tratou o registro faz a seguinte não ter o que fazer. Olhe a cadeia inteira.
- **Campo cru versus campo calculado.** Dois campos com nomes parecidos (o original e o corrigido) e uma parte do código usando o errado.

Mapa do meu sistema (preencha antes de usar)

A IA não adivinha onde estão as coisas no seu ambiente. Escreva uma vez:

Fonte	Onde fica no meu ambiente	Comando ou ferramenta
Versão deployada		
Banco de produção (leitura)		
Banco de homologação		
Logs		
Orquestrador / jobs		
Configuração		
CI		

E, toda vez que uma investigação revelar uma armadilha nova do seu sistema, acrescente uma linha em "Armadilhas comuns". É assim que a skill fica melhor que você em lembrar dos erros.

Quando NÃO usar

Erro local e reproduzível: stack trace na sua máquina, teste unitário falhando, erro de digitação, import quebrado. Ali a fonte da verdade é rodar o código. Esta skill é para comportamento de **sistema em produção**.

Credencial expirada

Se a autenticação cair no meio da investigação, peça para a pessoa renovar o login. Você não consegue completar o fluxo de autenticação sozinho, e **não substitua o dado real por estimativa** porque o acesso caiu.
